Table 5: Performance comparison of GlitchProber and Rule-based method on different models.

Model	Metric	Method	
		Rule-based Fix	GlitchProber
Llama-2-7b-chat	Repaired Tokens	3,805	4,021
	Repair Rate	59.22%	62.58%
Mistral-7B-Instruct-v0.1	Repaired Tokens	359	1,045
	Repair Rate	12.92%	37.60%
Qwen-7B-Chat	Repaired Tokens	10,645	14,765
	Repair Rate	34.68%	48.11%
Gemma-2b-it	Repaired Tokens	9,865	13,638
	Repair Rate	35.28%	48.77%
Yi-6B-Chat	Repaired Tokens	3,390	4,317
	Repair Rate	41.83%	53.26%
Average	Repaired Tokens	5,613	7,758
	Repair Rate	36.79%	50.06%

uses a fixed value to adjust activation values at the same neuron positions as GlitchProber.

Evaluation Metrics of Fix Experiments. We used two test metrics to evaluate the performance of the fix methods: the number of repaired glitch tokens (Repaired Tokens) and the repair rate (Repair Rate). The repair rate represents the proportion of glitch tokens successfully repaired out of all glitch tokens. It is calculated using the following formula:

$$\text{Repair Rate} = \frac{\text{Repaired Token Number}}{\text{Total Glitch Token Number}} \tag{13}$$

These two metrics can intuitively reflect the actual effectiveness of the fix methods.

Evaluation Settings. In our fix experiments, we remain the same $\gamma = 0.1$ for random sampling and choose the same key layers. For the threshold, we set $m = 1$ to determine $Neun^{\uparrow}$ and $Neun^{\downarrow}$.

6.2 RQ5 (Effective Fix): How effective is our approach in fixing glitch tokens across different LLMs?

We compared the performance of the GlitchProber fix algorithm with the rule-based fix method in terms of repaired tokens and repair rate under the same conditions to evaluate the effectiveness of GlitchProber. Table 5 presents the performance comparison of the two methods across different test models.

The results indicate that although the rule-based fix method uses fixed α and β values ($\alpha = 4$ and $\beta = 1.5$) and lacks flexibility, it can still direct the activation or inhibition of neurons associated with glitch tokens, achieving a certain degree of fix. This demonstrates that adjusting neuron activation values to correct the abnormal behavior of glitch tokens is a viable and effective fix strategy. However, due to its lack of specificity, the fix effect of this method is relatively limited. In contrast, GlitchProber precisely calculates α and β, and selectively adjusts the activation patterns of key feature neurons, achieving an average repair rate of 50.06% across the five models, outperforming the rule-based method.

Answer to RQ5

GlitchProber has effectively fixed glitch tokens across five LLMs. Compared to the rule-based method, its key improvement is the precise calculation of α and β, allowing better application to different models.

Table 6: Post process time of GlitchProber using various SVM parameter configurations (in seconds)

Feature Type	C=0.1 degree=2	C=0.5 degree=2	C=0.5 degree=3	C=1 degree=3
Attn_pattern	1,413.88	1,453.21	1,360.12	1,357.70
MLP_gate	1,407.36	1,458.88	1,436.23	1,445.11
MLP_data	1,473.43	1,541.10	1,579.08	1,581.00
Attn_pattern + MLP_gate	1,410.79	1,438.57	1,444.74	1,472.50
Attn_pattern + MLP_data	1,427.43	1,479.34	1,514.72	1,513.41
MLP_gate + MLP_data	1,453.72	1,504.36	1,546.09	1,530.88
Attn_pattern + MLP_gate + MLP_data	1,444.23	1,471.85	1,502.46	1,500.23

7 DISCUSSION

7.1 Hyperparameters Choice of GlitchProber

In this section, an experiment is conducted to illustrate the selection process for the principal features in LLMs and the hyperparameters in SVM. Specifically, the parameters C and $degree$ in the SVM's polynomial kernel require elucidation. The parameter C, commonly referred to as the regularization parameter, controls the trade-off between achieving a low error on the training data and minimizing the model complexity for better generalization to new data. A higher value of C tries to fit the training set as well as possible (higher model complexity), while a lower value leads to a model that might not perform as well on the training set but is better at generalizing. On the other hand, $degree$ pertains to the degree of the polynomial kernel function and is crucial for defining the complexity of the decision surface. A higher $degree$ results in more complex decision boundaries, capable of capturing more intricate patterns in the data. However, this also increases the risk of overfitting, particularly in scenarios with noise and limited data samples [33, 37].

As depicted in Table 6, no significant differences are observed in the time consumption across various groups of hyperparameters and features. Therefore, the average F1-score, as illustrated in Figure 6, is considered for comparison. The comparison of the F1-score without post-processing is chosen as it more accurately reflects the inherent effectiveness of the different features and hyperparameters. Figure 6 clearly shows that the hyperparameter group in the lower right corner achieves the highest F1-score of 0.6117 without post-processing, which is noteworthy. Consequently, $C = 1$ and degree = 3 are selected as the hyperparameters for SVM. All three features are chosen for the detection process, and two MLP-based features are chosen for the fix process.

We further examine the rationale behind the selection of the hyperparameter γ, which determines the sampling rate from the model's token vocabulary V for the sample set S in GlitchProber. The parameter γ is pivotal in balancing detection accuracy against computational efficiency. Empirical analysis, as shown in Table 7, demonstrates that a γ increment from 0.1 to 0.3 improves recall from 0.6922 to 0.7457 and raises the F1 score from 0.8181 to 0.8543, with a reasonable increase in computational time. However, further increasing γ to 0.7, while boosting recall and F1 scores to 0.7812 and 0.8772 respectively, significantly extends processing times to over 100 minutes. Therefore, a γ range of 0.1 to 0.3 is recommended, as it optimally balances performance gains with computational efficiency, ensuring that GlitchProber remains practical for operational use.

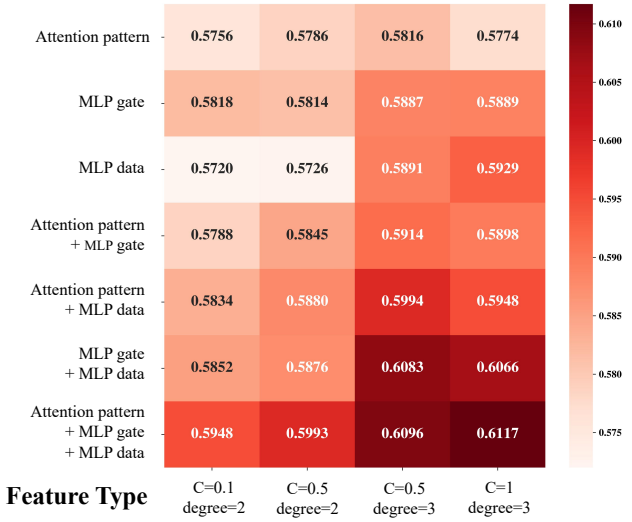

Figure 6: Different Feature Combination Comparison for GlitchProber

Table 7: Model performance across different gamma values on Llama2 model

Sampling Rate	$\gamma = 0.1$	$\gamma = 0.2$	$\gamma = 0.3$	$\gamma = 0.5$	$\gamma = 0.7$
Precision	100%	100%	100%	100%	100%
Recall	69.22%	72.81%	74.57%	76.65%	78.12%
F1 Score	81.81%	84.26%	85.43%	86.78%	87.72%
Time	61min 38s	68min 14s	74min 30s	86min 52s	100min 41s

Table 8: Performance comparison of original and modified model using Finetuning and GlitchProber for Mitigation

Dataset	Original Model	GlitchProber	Finetuning
GSM8K	0.315	0.301	0.238
HumanEval pass@1	0.129	0.103	0.009
HumanEval pass@5	0.190	0.161	0.024
MMLU	0.453	0.417	0.229

7.2 Rationale of GlitchProber versus Exhaustive Search

In the context of exhaustive search mechanisms, LLMs are required to generate, on average, **twenty** tokens at a zero temperature setting for every individual token processed. This computational method is both intensive and inefficient. Conversely, the implementation of GlitchProber necessitates merely a **single** forward processing step to capture the intermediate features of the LLM for each token. This approach substantially reduces 95% of redundant operations of those required by the traditional exhaustive search method.

7.3 Glitch Token Mitigation By Fine-tuning

GlitchProber adaptively modifies the process of model calculation without altering the model parameters, thereby minimizing the mitigation impact on model performance and preserving the basic abilities of LLMs. In contrast, we also construct a dataset with Q&A for the repetition task and attempt to mitigate the glitch token phenomenon by fine-tuning LLMs. However, compared with GlitchProber, fine-tuning LLMs alters the parameters of the model, potentially compromising its basic abilities. For example, we fine-tune the Llama-2-7b-chat with a dataset containing 3,000 Q&A pairs of repetition tasks. To evaluate the model's basic skills, we use three widely accepted datasets namely `GSM8K` [8], `HumanEval` [6], and `MMLU` [17].

Detailed results presented in Table 8 indicate that the model's ability in code writing and solving math problems post GlitchProber is comparable with the original model. and significantly better compared to the fine-tuned model, which notably diminished the basic abilities of original model.

7.4 Threats to Validity

Internal. Our primary concern involves the selection and computation of hyperparameters in both the detection and fixing phases of GlitchProber. For the detection phase, we detail the rationale behind our choices through an experiment described in Section 7.1. Various feature types and hyperparameters for the SVM consistently outperform the established baselines. In the fixing phase, the linear computation of α and β proves untenable. Enhanced discussion and manipulation of the activation value are recommended as future research directions.

External. Threats are associated with our experimental framework. For the performance of glitch tokens in intermediate layers, we experiment on three LLMs with different parameters and vocabulary sizes. Based on these findings, GlitchProber experiment on five different LLMs, showing its generalizability. Furthermore, as GlitchProber is required to access the intermediate data of the LLM, GlitchProber is only applicable to open-source LLMs. The transferability of GlitchProber from open-source LLMs to closed-source LLMs like GPT-4 could further be explored.

8 CONCLUSION

In this work, we reveal that glitch tokens trigger abnormal activation characteristics in the model's attention patterns and MLP status through systematic empirical study. Inspired by this, we propose methods for detecting and fixing glitch tokens. GlitchProber employs a sampling strategy, extracting features from attention patterns and MLP status, and achieves rapid screening of the vocabulary through PCA dimensionality reduction and SVM classification. Experiments on LLMs demonstrate that GlitchProber saves 40% of the time compared to existing methods while achieving higher accuracy. Another important contribution is our strategy to fix glitch tokens by adjusting the activation values of the model's intermediate layers. Experiments on LLMs confirm the effectiveness of this fix strategy, and the average repair rate of GlitchProber was improved by 13.27% compared with the baseline method.

ACKNOWLEDGEMENT

This work was supported by the National NSF of China (grants No.62302176, No.62072046, 62302181), the Key R&D Program of Hubei Province (2023BAB017, 2023BAB079), and the Knowledge Innovation Program of Wuhan-Basic Research (2022010801010083).

REFERENCES

[1] Hervé Abdi and Lynne J Williams. 2010. Principal component analysis. *Wiley interdisciplinary reviews: computational statistics* 2, 4 (2010), 433–459.
[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal

Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

[3] 01. AI, :, Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Heng Li, Jiangcheng Zhu, Jianqun Chen, Jing Chang, Kaidong Yu, Peng Liu, Qiang Liu, Shawn Yue, Senbin Yang, Shiming Yang, Tao Yu, Wen Xie, Wenhao Huang, Xiaohui Hu, Xiaoyi Ren, Xinyao Niu, Pengcheng Nie, Yuchi Xu, Yudong Liu, Yue Wang, Yuxuan Cai, Zhenyu Gu, Zhiyuan Liu, and Zonghong Dai. 2024. Yi: Open Foundation Models by 01.AI. arXiv:2403.04652 [cs.CL]

[4] Anthropic. 2024. The Claude 3 Model Family: Opus, Sonnet, Haiku. Online. https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf (Accessed: 2024-04-24).

[5] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen Technical Report. arXiv:2309.16609 [cs.CL]

[6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG]

[7] Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, et al. 2020. Rethinking attention with performers. *arXiv preprint arXiv:2009.14794* (2020).

[8] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).

[9] Corinna Cortes and Vladimir Vapnik. 1995. Support-vector networks. *Machine learning* 20, 3 (1995), 273–297.

[10] Gelei Deng, Yi Liu, Yuekang Li, Kailong Wang, Ying Zhang, Zefeng Li, Haoyu Wang, Tianwei Zhang, and Yang Liu. 2024. MasterKey: Automated Jailbreak Across Multiple Large Language Model Chatbots. In *NDSS*.

[11] Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, and Yang Liu. 2024. Pandora: Jailbreak GPTs by Retrieval Augmented Generation Poisoning. *NDSS AISCC* (2024).

[12] Martin Fell. 2023. A Search for More ChatGPT / GPT-3.5 / GPT-4 "Unspeakable" Glitch Tokens. Online. https://www.lesswrong.com/posts/kmWrwtGE9B9hpbgRT/a-search-for-more-chatgpt-gpt-3-5-gpt-4-unspeakable-glitch (Accessed: 2024-05-05).

[13] Jonas Geiping, Alex Stein, Manli Shu, Khalid Saifullah, Yuxin Wen, and Tom Goldstein. 2024. Coercing LLMs to do and reveal (almost) anything. *arXiv preprint arXiv:2402.14020* (2024).

[14] Thomas Mesnard Gemma Team, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, and et al. 2024. Gemma. (2024). https://doi.org/10.34740/KAGGLE/M/3301

[15] GlitchProber. (Accessed on 06/07/2024). https://sites.google.com/view/glitchprober/.

[16] Hao Guan, Guangdong Bai, and Yepang Liu. 2024. Large Language Models Can Connect the Dots: Exploring Model Optimization Bugs with Domain Knowledge-Aware Prompts. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis* (Vienna, Austria) *(ISSTA 2024)*. Association for Computing Machinery, New York, NY, USA, 1579–1591. https://doi.org/10.1145/3650212.3680383

[17] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300* (2020).

[18] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL]

[19] Haodong Li, Gelei Deng, Yi Liu, Kailong Wang, Yuekang Li, Tianwei Zhang, Yang Liu, Guoai Xu, Guosheng Xu, and Haoyu Wang. 2024. Digger: Detecting Copyright Content Mis-usage in Large Language Model Training.

[20] Ningke Li, Yuekang Li, Yi Liu, Ling Shi, Kailong Wang, and Haoyu Wang. 2024. Drowzee: Metamorphic Testing for Fact-conflicting Hallucination Detection in Large Language Models. In *OOPSLA (To Appear)*.

[21] Yuxi Li, Yi Liu, Gelei Deng, Ying Zhang, Wenjia Song, Ling Shi, Kailong Wang, Yuekang Li, Yang Liu, and Haoyu Wang. 2024. Glitch Tokens in Large Language Models: Categorization Taxonomy and Effective Detection. In *FSE*.

[22] Yuxi Li, Yi Liu, Yuekang Li, Ling Shi, Gelei Deng, Shengquan Chen, and Kailong Wang. 2024. Lockpicking LLMs: A Logit-Based Jailbreak Using Token-level Manipulation.

[23] Hanxiao Liu, Zihang Dai, David R. So, and Quoc V. Le. 2021. Pay Attention to MLPs. arXiv:2105.08050 [cs.LG]

[24] Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2023. Prompt Injection attack against LLM-integrated Applications. *arXiv preprint arXiv:2306.05499* (2023).

[25] Haoneng Luo, Shiliang Zhang, Ming Lei, and Lei Xie. 2020. Simplified self-attention for transformer-based end-to-end speech recognition. *arXiv preprint arXiv:2005.10463* (2020).

[26] mwatkins. 2023. The petertodd phenomenon. Online. https://www.lesswrong.com/posts/jkY6QdCfAXHJk3kea/the-petertodd-phenomenon (Accessed: 2024-05-05).

[27] mwatkins. 2023. A Search for More ChatGPT/GPT-3.5/GPT-4 "Unspeakable" Glitch Tokens. Online. https://www.lesswrong.com/posts/kmWrwtGE9B9hpbgRT/a-search-for-more-chatgpt-gpt-3-5-gpt-4-unspeakable-glitch (Accessed: 2024-05-03).

[28] mwatkins and Jessica Rumbelow. 2023. SolidGoldMagikarp II: technical details and more recent findings. Online. https://www.lesswrong.com/posts/Ya9LzwEbfaAMY8ABo/solidgoldmagikarp-ii-technical-details-and-more-recent (Accessed: 2024-05-05).

[29] mwatkins and Jessica Rumbelow. 2023. SolidGoldMagikarp III: Glitch token archaeology. Online. https://www.lesswrong.com/posts/8viQEp8KBg2QSW4Yc/solidgoldmagikarp-iii-glitch-token-archaeology (Accessed: 2024-05-05).

[30] Bloom J Nanda N. 2022. TransformerLens. Online. https://github.com/neelnanda-io/TransformerLens (Accessed: 2024-05-05).

[31] Machel Reid, Nikolay Savinov, Denis Teplyashin, Dmitry Lepikhin, Timothy Lillicrap, Jean-baptiste Alayrac, Radu Soricut, Angeliki Lazaridou, Orhan Firat, Julian Schrittwieser, et al. 2024. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).

[32] Jessica Rumbelow and mwatkins. 2023. SolidGoldMagikarp(plus, prompt generation). Online. https://www.lesswrong.com/posts/aPeJE8bSo6rAFoLqg/solidgoldmagikarp-plus-prompt-generation (Accessed: 2024-05-05).

[33] Bernhard Scholkopf and Alexander J. Smola. 2002. *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT Press.

[34] Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).

[35] Oguzhan Topsakal and Tahir Cetin Akinci. 2023. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International Conference on Applied Engineering and Natural Sciences*, Vol. 1. 1050–1056.

[36] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open Foundation and Fine-Tuned Chat Models. arXiv:2307.09288 [cs.CL]

[37] Vladimir N. Vapnik. 1995. *The Nature of Statistical Learning Theory*. Springer-Verlag.

[38] Leonid Nisonovich Vaserstein. 1969. Markov processes over denumerable products of spaces, describing large systems of automata. *Problemy Peredachi Informatsii* 5, 3 (1969), 64–72.

[39] Dixuan Wang, Yanda Li, Junyuan Jiang, Zepeng Ding, Guochao Jiang, Jiaqing Liang, and Deqing Yang. 2024. Tokenization Matters! Degrading Large Language Models through Challenging Their Tokenization. *arXiv preprint arXiv:2405.17067* (2024).

[40] Guanyu Wang, Yuekang Li, Yi Liu, Gelei Deng, Tianlin Li, Guosheng Xu, Yang Liu, Haoyu Wang, and Kailong Wang. 2024. MeTMaP: Metamorphic Testing for Detecting False Vector Matching Problems in LLM Augmented Generation. *FORGE* (2024).